ภาคผนวก ง

ซอร์สโค้ดคลาสการทำงานของ Pre – processing และ Feature Extraction

```
class PROCESS
{
	private:
		Matrix MELFILTERBANK(double p,double n,double fs);
		Matrix HAMMING(int n);
		Matrix DCT(Matrix x,double n);
		Matrix FFT_DCT(Matrix b,Matrix m,int order);
		Matrix MFCC(Matrix s,double fs);
	public:
		Matrix EXTRACT(Matrix wavtrain,int length_wav,int
		no_cmd);
		Matrix ENDPOINT(Matrix m);
};
//*************************************************************************
```

Pre – Processing

```
//*************************************************************************
Matrix PROCESS ::ENDPOINT(Matrix m){

	Matrix wrap(44100,1);

	for (int a=0;a<44100;a++){		//initialize temp
		wrap(a,0) = 0.000763;
	}

	if(m.RowNo() < 41000){
		for (a=0;a<m.RowNo();a++){	//matrix m to matrix wrap
			wrap(a,0) = m(a,0);
		}
	}else{
		for (a=0;a<44100;a++){
		wrap(a,0) = m(a,0);
		}
	}

	float max = 0;
	int p_max = 0;

	for (a=0;a<44100;a++){		// find MAX ampitude in wave
		if (abs(wrap(a,0)) > max){
			max = abs(wrap(a,0));
			p_max = a;
		}
	}

	float cut_point = 0.2*max;	// define threshold valur

	a = 0;

	while (abs(wrap(a,0)) < cut_point){
		a++;
	}

	int st;
	st = a;				// start point of wave

	Matrix temp(10000,1);		// creat matrix has lenght 10000
	int x = 0;
	int y = 0;
```

```
	for (int a2 = st ; a2 < st+10000 ; a2++){
		temp(x,y) = wrap(a2,0);
		x++;
	}
	return temp;
}
//*****************************************************************************
```

Feature Extraction

```
//*****************************************************************************
//------------------------------- MFCC ------------------------------
Matrix PROCESS ::MFCC(Matrix s,double fs) {
	int N = 1024;	// N = pow(2,order)
	int M = N/2;
	int nof = 40;
	int length,cols,row,col;
	double fin;

	length = s.RowNo();
	fin = length-N+1;
	cols = floor(fin/(N-M));

	Matrix a;
	a.Null(N,cols);

	for (row=0; row<N; row++) {
		a(row,0) = s(row,0);
	}

	for (col=1; col<cols; col++) {
		for (row=0; row<N; row++) {
			a(row,col) = s((N-M)*col+row,0);
		}
	}
	//------ Hamming Window -------
	Matrix h(N,1),b(N,cols);
	h = HAMMING(N);

	for (col=0; col<cols; col++) {
		for (row=0; row<N; row++) {
			b(row,col) = a(row,col)*h(row,0);
		}
	}
	//------ Melfilterbank ------
	Matrix m;
	m = MELFILTERBANK(nof,N,fs);

	Matrix v;
	int order = 0;

	while ( N != pow(2,order) ) {
		order++;
	}

	v = FFT_DCT(b,m,order);

	return v;
}
```

```
//-------------------------- Hamming Window -------------------------
Matrix PROCESS ::HAMMING(int n){
      double arg;
      int t;
      Matrix h(n,1);

      arg = 2.0*PI/(n-1);
      t = n/2;

      for (int j = 0; j < n; j++) {
            h(j,0) = 0.54 - 0.46*cos(arg*j);
      }
      return h;
}
//-------------------------- Melfilterbank -------------------------
Matrix PROCESS ::MELFILTERBANK(double p,double n,double fs){
      double f0 , fn2 , lr;
      double b1,b2,b3,b4;
      int i;
      Matrix a(4,1);

      f0 = 700/fs;
      fn2 = floor(n/2);
      lr = log(1 + 0.5/f0) / (p+1);

      a(0,0) = 0;
      a(1,0) = 1;
      a(2,0) = p;
      a(3,0) = p+1;
      for (i = 0; i<4; i++) {
            a(i,0) = n*(f0*(exp(a(i,0)*lr)-1));
      }
      b1 = floor(a(0,0)) + 1;
      b2 = ceil(a(1,0));
      b3 = floor(a(2,0));
      b4 = __min(fn2,ceil(a(3,0)))-1;

      Matrix pf(1,b4),fp(1,b4),pm(1,b4);
      for (i=b1; i<=b4; i++) {
            pf(0,i-1) = log(1 + (i/n/f0)) / lr;
            fp(0,i-1) = floor(pf(0,i-1));
            pm(0,i-1) = pf(0,i-1) - fp(0,i-1);
      }

      int cols = b3+b4-b2+b1;
      int temp=0;
      int row,col;
      Matrix r(1,cols),c(1,cols),v(1,cols),m(p,1+fn2);

      for (col=b2; col<=b4; col++) {
            r(0,col - b2) = fp(0,col-1);
      }

      for (col=1; col<=b3; col++) {
            temp = b4+col-b2;
            r(0,temp) = 1 + fp(0,col-1);
      }

      for (col=b2; col<=b4; col++) {
            c(0,col-b2) = col + 1;
      }
```

```
	for (col=1; col<=b3; col++) {
		temp = b4 - b2 + col ;
		c(0,temp) = col + 1;
	}

	for (col=b2; col<=b4; col++) {
		v(0,col-b2) = 2*(1 - pm(0,col-1));
	}

	for (col=1; col<=b3; col++) {
		temp = b4 - b2 + col;
		v(0,temp) = 2*(pm(0,col-1));
	}

	m.Null(p,1+fn2);
	for (i=0; i<b3+b4-b2+b1; i++) {
		row = r(0,i)-1;
		col = c(0,i)-1;
		m(row,col) = v(0,i);
	}

	return m;
}
// ------------------------------ FFT -------------------------------
Matrix PROCESS ::FFT_DCT(Matrix b,Matrix m,int order) {
	int rows = b.RowNo();
	int N = rows;
	int cols = b.ColNo();
	int r=0,c=0;
	int n2;

	n2 = 1+floor(N/2);

	Matrix y(n2,1);
	Matrix abs_y(n2,1);
	Matrix ms(m.RowNo(),1);
	Matrix log_ms(ms.RowNo(),1);
	Matrix dct_o(log_ms.RowNo(),1);
	Matrix v(dct_o.RowNo()-2,cols);

	double temp = 0;
	double* re = new double[rows];
	double* im = new double[rows];

	for(c=0; c<cols; c++) {
		for (r=0; r<rows; r++){
			re[r] = b(r,c);
			im[r] = 0.0;
		}

		fft(re,im,order,FFT);

		for (r=0; r<n2; r++) {
			y(r,0) = sqrt(pow(re[r],2) + pow(im[r],2));
		}

		abs_y = y ^ 2;

		ms = m * abs_y;
```

```
            for (r=0; r<ms.RowNo(); r++) {
                  log_ms(r,0) = log(ms(r,0));
            }

            dct_o = DCT(log_ms,log_ms.RowNo());

            // ----------- Get to v (Cut row 0,1)--------------
            for (r=0; r<dct_o.RowNo()-2; r++) {
                  v(r,c) = dct_o(r+2,0);
            }
      }
      delete[] re; //--- Don't forget
      delete[] im; //--- Don't forget

      return v;
}
// ------------------------------ DCT -----------------------------
Matrix PROCESS ::DCT(Matrix x,double n) {
      long double sum,angle;
      double k = n;
      Matrix a(x.RowNo(),x.ColNo());

      for (int i=0; i<k; i++) {
            sum = 0;
            for(int j=0; j<n; j++) {
                  angle = ((PI)*((2*(j+1))-1)*((i+1)-1))/(2*n);
                  sum = sum + (x(j,0)*cos(angle));
            }
            if (i==0) {
                  a(i,0) = (1/(sqrt(n)))*sum;
            }
            else {
                  a(i,0) = (sqrt(2/n))*sum;
            }

      }
      return a;
}
// ---------------------------------------------------------------------
```